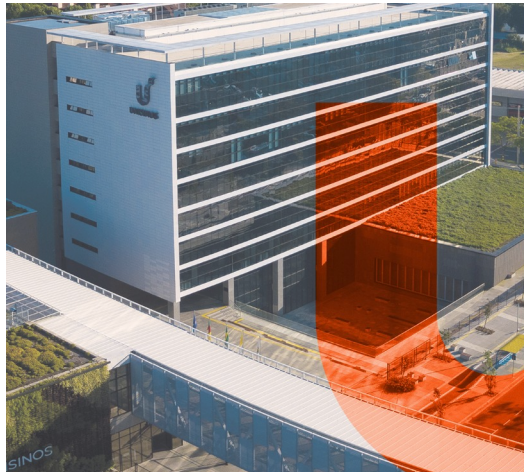




Síntese: Dados

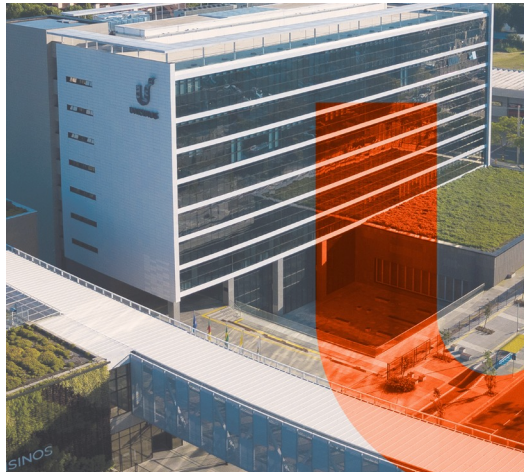
Dados Semiestruturados e JSON

Ao longo desta atividade acadêmica, percorremos as etapas de construção de um banco de dados relacional: modelagem conceitual, projeto lógico, normalização, álgebra relacional e SQL. Nesta última semana de conteúdo, ampliamos essa visão para além das tabelas, conhecendo os dados semiestruturados e o formato JSON, presentes em praticamente toda aplicação moderna. Esta síntese retoma os pontos centrais desse estudo e os conecta ao que construímos nas semanas anteriores.

1.1 Três formas de organizar os dados

Os dados podem ser classificados conforme o rigor de sua organização. Os **dados estruturados** seguem um esquema rígido definido antes da escrita (schema-on-write): é o mundo dos bancos relacionais, em que cada registro precisa obedecer aos tipos e às restrições de suas colunas. Os **dados semiestruturados** não exigem esquema fixo, mas carregam sua própria organização por meio de marcadores e chaves autodescritivas (schema-on-read), como nos arquivos JSON, XML e YAML. Já os **dados não estruturados**, como imagens, vídeos, áudios e textos livres, não possuem estrutura formalizável e exigem ferramentas analíticas especializadas.

Nenhuma dessas formas é superior às demais. A escolha depende dos requisitos da aplicação, da taxa de variação estrutural dos dados e das necessidades de desempenho da solução.



Síntese: Dados

1.2 O formato JSON

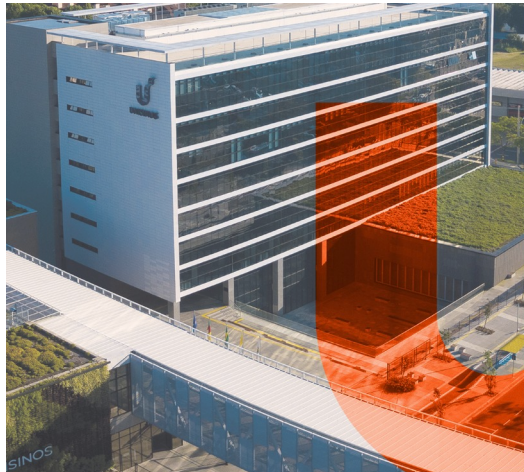
O **JSON** (JavaScript Object Notation) é um padrão aberto, definido pela RFC 8259, independente de linguagem de programação. Sua estrutura é baseada em **pares chave-valor**: as chaves, sempre entre aspas duplas, identificam os atributos, e os valores carregam os dados. O formato reconhece nativamente os tipos primitivos string, number, boolean e null, além de dois tipos complexos: o **objeto**, delimitado por chaves { }, e o **array**, delimitado por colchetes [].

Objetos aninhados e arrays permitem representar estruturas hierárquicas completas em um único documento. No estudo de caso da Companhia Aérea, um documento de voo pode encapsular os dados da aeronave e a lista da equipe de pilotos, informações que, no modelo relacional, estariam distribuídas em tabelas conectadas por chaves estrangeiras.

1.3 JSON, XML e YAML: cada formato no seu lugar

Os três principais formatos semiestruturados têm vocações distintas. O **XML** é extensível e possui validação formal robusta, mas é verboso; permanece forte em sistemas legados, barramentos SOAP e integrações governamentais como a NF-e. O **YAML** prioriza a legibilidade humana e se consolidou nos arquivos de configuração e de infraestrutura como código. O **JSON**, leve e de processamento simples, tornou-se o formato predominante no tráfego de dados da web, conectando clientes e servidores por meio de APIs REST.

Um ponto merece destaque: o formato de tráfego de uma API não determina o formato de armazenamento. Um servidor pode manter seus dados normalizados em um banco relacional e serializá-los para JSON apenas no momento de responder às requisições.



Síntese: Dados

1.4 Bancos de dados orientados a documentos

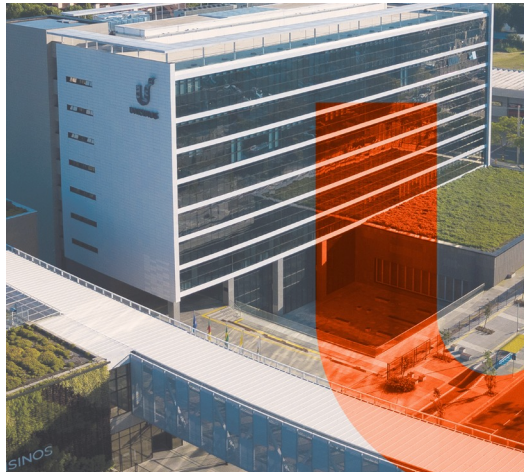
A consolidação do JSON impulsionou os bancos **NoSQL orientados a documentos**, em que a unidade de armazenamento deixa de ser a linha de uma tabela e passa a ser o próprio **documento**, geralmente persistido em formato binário otimizado (BSON). Documentos semelhantes são agrupados em **coleções**, e a escalabilidade ocorre de forma horizontal, distribuindo os dados em clusters (sharding). Entre os principais representantes estão o MongoDB, o CouchDB e o Firebase Firestore.

Esses sistemas favorecem a leitura rápida e o esquema dinâmico: todas as informações de uma entidade podem ser recuperadas em uma única operação, sem junções, e novos atributos podem ser incluídos sem reestruturar o banco. O custo é a desnormalização: dados duplicados em muitos documentos geram redundância e criam risco de anomalias de atualização, exatamente os problemas que a normalização estudada nas semanas anteriores busca evitar.

1.5 Quando usar cada abordagem

A decisão entre o modelo relacional e o modelo de documentos é uma análise de perdas e ganhos. Cenários que exigem consistência transacional estrita, como sistemas bancários, continuam sendo território dos bancos relacionais e de suas propriedades **ACID**. Cenários de alta variabilidade estrutural e grande volume de escrita, como a telemetria de sensores IoT ou os catálogos de streaming e e-commerce, favorecem os documentos semiestruturados.

Na prática, as abordagens coexistem. A **persistência poliglota** consiste em usar tecnologias de armazenamento diferentes dentro do mesmo projeto, cada uma no cenário em que apresenta melhor adequação: dados financeiros no relacional, catálogos flexíveis em documentos.



Síntese: Dados

Conhecer os dados semiestruturados não substitui o que estudamos até aqui: amplia. A análise de requisitos, a modelagem das entidades e a compreensão das cardinalidades continuam guiando o projeto, seja qual for o repositório final. O que muda é a forma física de representar e armazenar o que foi modelado. Dominar os dois paradigmas permite escolher, com critério técnico, a ferramenta certa para cada problema.